# DATABASE SYSTEMS

## Laboratory Work: Database Constraints in PostgreSQL

**Student Name:** ASHIMZHANULY BEKARYS
**Student ID:** 25B031243

### Objective

The objective of this laboratory work is to learn how to use database constraints in PostgreSQL. The work covers CHECK, NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY and ON DELETE constraints. It also includes the creation of a simple e-commerce database.

---

## Task 1. CHECK Constraint

**Objective:** To restrict the values that can be inserted into a table.

**SQL Code:**

```
CREATE TABLE employees (
    employee_id INTEGER,
    first_name TEXT,
    last_name TEXT,
    age INTEGER CHECK (age BETWEEN 18 AND 65),
    salary NUMERIC CHECK (salary > 0)
);
```

**Explanation:**

The CHECK constraint controls the values entered into the table. The employee's age must be between 18 and 65, and the salary must be greater than zero.

**Test:**

```
SELECT * FROM employees;
-- This insert should fail because the age is below 18.
-- INSERT INTO employees
-- VALUES (2, 'Test', 'User', 15, 200000);

-- This insert should fail because the salary is negative.
-- INSERT INTO employees
-- VALUES (3, 'Test', 'User', 25, -1000);
```

**Result:** The employees table was created successfully. The CHECK constraints are defined to reject invalid age and salary values.

---

## Task 2. NOT NULL Constraint

**Objective:** To prevent required fields from containing NULL values.

**SQL Code:**

```
CREATE TABLE employees_not_null (
    employee_id INTEGER,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    age INTEGER NOT NULL
);
```

**Insert valid data:**

```
INSERT INTO employees_not_null
VALUES (1, 'Bekarys', 'Ashimzhanuly', 18);

SELECT * FROM employees_not_null;
```

**Test:**

```
-- This insert should fail because first_name is NULL.
-- INSERT INTO employees_not_null
-- VALUES (2, NULL, 'Test', 20);
```

**Explanation:**

The NOT NULL constraint requires a value in the specified column. In this example, the first name, last name and age cannot be NULL.

**Result:** The table is designed to accept valid employee data and reject records with missing required values.

---

# Task 3. UNIQUE Constraint

**Objective:** To prevent duplicate values in a column.

**SQL Code:**

```
CREATE TABLE users (
    user_id INTEGER,
    username TEXT UNIQUE,
    email TEXT UNIQUE
);
```

**Insert valid data:**

```
INSERT INTO users
VALUES (1, 'bekarys01', 'bekarys01@example.com');

INSERT INTO users
VALUES (2, 'student02', 'student02@example.com');

SELECT * FROM users;
```

**Test:**

```
-- This insert should fail because the email already exists.
-- INSERT INTO users
-- VALUES (3, 'student03', 'bekarys01@example.com');
```

**Explanation:**

The UNIQUE constraint does not allow two records to have the same username or email address. This helps prevent duplicate account information.

**Result:** The table is designed to reject duplicate usernames and email addresses.

---

# Task 4. PRIMARY KEY Constraint

**Objective:** To identify each record uniquely.

**SQL Code:**

```
CREATE TABLE departments (
    department_id INTEGER PRIMARY KEY,
    department_name TEXT NOT NULL
);
```

**Insert data:**

```
INSERT INTO departments
VALUES
    (1, 'IT'),
    (2, 'Finance'),
    (3, 'Marketing');

SELECT * FROM departments;
```

**Test:**

```
-- This insert should fail because department_id 1 already exists.
-- INSERT INTO departments
-- VALUES (1, 'Human Resources');
```

**Explanation:**

A PRIMARY KEY uniquely identifies every record in a table. It does not allow duplicate or NULL key values.

**Result:** The departments table uses department_id as its primary key.

---

# Task 5. FOREIGN KEY and ON DELETE Constraints

**Objective:** To connect tables and maintain referential integrity.

**SQL Code:**

```
CREATE TABLE department_employees (
    employee_id INTEGER PRIMARY KEY,
    employee_name TEXT NOT NULL,
    department_id INTEGER,
    FOREIGN KEY (department_id)
        REFERENCES departments(department_id)
        ON DELETE SET NULL
);
```

**Insert data:**

```
INSERT INTO department_employees
VALUES
    (1, 'Bekarys', 1),
    (2, 'Student Two', 2),
    (3, 'Student Three', 3);

SELECT * FROM department_employees;
```

**Test:**

```
-- This insert should fail because department 99 does not exist.
-- INSERT INTO department_employees
-- VALUES (4, 'Student Four', 99);
```

**Explanation:**

The FOREIGN KEY connects department_employees to departments. The department_id value must refer to an existing department.

ON DELETE SET NULL means that when a referenced department is deleted, the corresponding department_id in department_employees becomes NULL. The employee record remains in the table.

**Result:** The foreign key maintains the relationship between the two tables and prevents references to nonexistent departments.

---

# Task 6. E-commerce Database Schema

**Objective:** To create a simple online shopping database with customers, products, orders and order details.

## 6.1. Customers Table

```
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    full_name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    city TEXT NOT NULL
);
```

**Explanation:** This table stores customer information. Each customer has a unique ID and email address.

## 6.2. Products Table

```
CREATE TABLE products (
    product_id INTEGER PRIMARY KEY,
    product_name TEXT NOT NULL,
    price NUMERIC(10, 2) NOT NULL CHECK (price >= 0),
    stock INTEGER NOT NULL CHECK (stock >= 0)
);
```

**Explanation:** This table stores product names, prices and available stock. Prices and stock values cannot be negative.

## 6.3. Orders Table

```
CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    order_date DATE NOT NULL,
    status TEXT NOT NULL
        CHECK (status IN (
            'pending',
            'processing',
            'shipped',
            'delivered',
            'cancelled'
        )),
    FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
        ON DELETE CASCADE
);
```

**Explanation:** Each order belongs to an existing customer. The status must be one of the allowed values. ON DELETE CASCADE deletes a customer's orders if that customer is deleted.

## 6.4. Order Details Table

```
CREATE TABLE order_details (
    order_detail_id INTEGER PRIMARY KEY,
    order_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    unit_price NUMERIC(10, 2) NOT NULL
        CHECK (unit_price >= 0),
    FOREIGN KEY (order_id)
        REFERENCES orders(order_id)
        ON DELETE CASCADE,
    FOREIGN KEY (product_id)
        REFERENCES products(product_id)
        ON DELETE RESTRICT
);
```

**Explanation:** This table stores the products included in each order. Quantity must be greater than zero. If an order is deleted, its details are deleted automatically. A product cannot be deleted while it is referenced by an order detail.

## 6.5. Insert Sample Data

**Customers:**

```
INSERT INTO customers VALUES
(1, 'Bekarys Ashimzhanuly', 'bekarys@example.com', 'Almaty'),
(2, 'Ayan Sadykov', 'ayan@example.com', 'Astana'),
(3, 'Diana Karimova', 'diana@example.com', 'Shymkent'),
(4, 'Arman Nurlanov', 'arman@example.com', 'Aktobe'),
(5, 'Aliya Serikova', 'aliya@example.com', 'Taraz');
```

**Products:**

```
INSERT INTO products VALUES
(1, 'Laptop', 350000.00, 10),
(2, 'Mouse', 5000.00, 50),
(3, 'Keyboard', 12000.00, 30),
(4, 'Monitor', 80000.00, 15),
(5, 'Headphones', 18000.00, 25);
```

**Orders:**

```
INSERT INTO orders VALUES
(1, 1, '2026-10-01', 'pending'),
(2, 2, '2026-10-02', 'processing'),
(3, 3, '2026-10-03', 'shipped'),
(4, 4, '2026-10-04', 'delivered'),
(5, 5, '2026-10-05', 'cancelled');
```

**Order Details:**

```
INSERT INTO order_details VALUES
(1, 1, 1, 1, 350000.00),
(2, 2, 2, 2, 5000.00),
(3, 3, 3, 1, 12000.00),
(4, 4, 4, 1, 80000.00),
(5, 5, 5, 2, 18000.00);
```

## 6.6. Verify the Data

```
SELECT * FROM customers;
SELECT * FROM products;
SELECT * FROM orders;
SELECT * FROM order_details;
```

**Explanation:**

The four tables store customer information, product information, orders and the products included in each order. Primary keys identify records, foreign keys connect the tables, and CHECK, NOT NULL and UNIQUE constraints validate the data.

Each table contains five sample records.

## 6.7. Test the Constraints

```
-- Duplicate customer email: should fail.
-- INSERT INTO customers
```

```
-- VALUES (6, 'Test Customer', 'bekarys@example.com', 'Almaty');

-- Negative product price: should fail.
-- INSERT INTO products
-- VALUES (6, 'Test Product', -100.00, 5);

-- Negative stock: should fail.
-- INSERT INTO products
-- VALUES (7, 'Test Product', 100.00, -5);

-- Invalid order status: should fail.
-- INSERT INTO orders
-- VALUES (6, 1, '2026-10-06', 'unknown');

-- Zero quantity: should fail.
-- INSERT INTO order_details
-- VALUES (6, 1, 1, 0, 350000.00);

-- Nonexistent customer: should fail.
-- INSERT INTO orders
-- VALUES (7, 99, '2026-10-06', 'pending');
```

**Explanation:**

These tests demonstrate how the database prevents invalid data from being inserted. The commands are commented out so that they do not interrupt the main script. They can be tested separately by removing the comment markers.

---

# Conclusion

In this laboratory work, I learned how to use database constraints in PostgreSQL. I created tables using CHECK, NOT NULL, UNIQUE, PRIMARY KEY and FOREIGN KEY constraints. I also learned how ON DELETE actions maintain relationships between tables.

Finally, I created an e-commerce database consisting of customers, products, orders and order_details. I added sample data and prepared queries to test the constraints.

These constraints help maintain data accuracy, consistency and integrity in a relational database.